



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto: Sistema Verificador de Cumplimiento OWASP

Curso: Calidad Y Pruebas De Software

Docente: Mag. Ing. Patrick Cuadros Quiroga

Integrantes:

Concha Llaca, Gerardo Alejandro

(2017057849)

Diego Fabrizio Andia Navarro

(2022073906)

Tacna – Perú

2026



CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
07	Japura Quispe, Herminia Aurelia Concha Llaca, Gerardo Alejandro	Mag. Ricardo Eduardo Valcarcel Alvarado	Mag. Ricardo Eduardo Valcarcel Alvarado	15/09/25	

***Sistema Verificador de Cumplimiento OWASP
Documento de Especificación de Requerimientos de Software***

Versión {1.0}



ÍNDICE GENERAL

I. Generalidades de la Empresa	4
1. Nombre de la Empresa	4
2. Visión	4
3. Misión	4
4. Organigrama	4
II. Visionamiento de la Empresa	5
1. Descripción del Problema	5
2. Objetivos	5
3. Alcance del proyecto	6
4. Viabilidad del Sistema	6
5. Información obtenida del Levantamiento de Información	7
III. Análisis de Procesos	7
a) Diagrama del Proceso Actual – Diagrama de actividades	7
b) Diagrama del Proceso Propuesto – Diagrama de actividades Inicial:	8
IV. Especificación de Requerimientos de Software	9
a) Cuadro de Requerimientos funcionales Inicial	9
b) Cuadro de Requerimientos No funcionales	10
c) Cuadro de Requerimientos funcionales Final:	11
d) Reglas de Negocio:	15
V. Fase de Desarrollo	16
1. Perfiles de Usuario	16
2. Modelo Conceptual	18
a) Diagrama de Paquetes:	18
b) Diagrama de Casos de Uso	19
c) Escenarios de Caso de Uso (narrativa)	25
3. Modelo Lógico	36
a) Análisis de Objetos	36
b) Diagrama de Error! Bookmark not defined. c)Diagrama de Secuencia	45
4. Diagrama de Clases:	50

I. Generalidades de la Empresa



1. Nombre de la Empresa

Securify Software Solutions

2. Visión

Ser la consultora de ciberseguridad y desarrollo seguro de software de referencia a nivel nacional, reconocida por el diseño y provisión de herramientas automatizadas de auditoría estática (SAST) que se integren de forma transparente en el flujo de trabajo diario de los desarrolladores.

3. Misión

Mitigar los riesgos de ciberseguridad en el ciclo de vida del software de nuestros clientes mediante la provisión de herramientas libres de análisis estático que evalúen el estándar OWASP Top 10 y vulnerabilidades de dependencias (CVEs) en tiempo real, capacitando a los equipos de desarrollo para entregar código robusto y de alta calidad.

4. Organigrama

Figura 1: Organigrama de la empresa.





II. Visionamiento de la Empresa

1. Descripción del Problema

Los desarrolladores de software modernos se enfrentan a constantes ciberamenazas que vulneran la privacidad y estabilidad de los sistemas. A pesar de existir marcos de seguridad como OWASP Top 10, la falta de retroalimentación inmediata durante la codificación provoca que se introduzcan vulnerabilidades básicas de manera involuntaria.

Los problemas críticos son:

- Exposición de secretos: Inclusión de llaves API, tokens y credenciales en texto plano dentro del código.
- Funciones inseguras: El uso de rutinas vulnerables a inyección (``eval``, ``exec``).
- Dependencias vulnerables: Incorporación de librerías desactualizadas con fallos públicos de seguridad (CVEs).
- Malas configuraciones de red: Despliegue de endpoints web sin cabeceras HTTP de protección (CSP, HSTS).

2. Objetivos

1.1.Objetivo General

Diseñar e implementar un sistema verificador que permita evaluar de manera ágil y en tiempo real el cumplimiento de los controles de seguridad OWASP Top 10 en código, repositorios y URLs, integrando interfaces web, APIs y entornos de desarrollo (IDE).



1.2.Objetivos específicos

- Diseñar una plataforma accesible y fácil de usar (Dashboard Web y Extensión IDE) que permita a los desarrolladores realizar análisis estáticos de código de forma rápida e intuitiva.
- Incorporar un motor de análisis estático basado en expresiones regulares y análisis de dependencias (CVE) que alerte al desarrollador sobre fallos de seguridad críticos del OWASP Top 10.
- Proveer a los administradores y auditores con una herramienta centralizada que les permita gestionar usuarios, crear y revocar tokens de API, y realizar el monitoreo histórico del log de accesos de red.
- Garantizar la portabilidad y accesibilidad desde diferentes entornos y plataformas (Windows, Linux y macOS) sin requerir infraestructuras de base de datos complejas.
- Implementar un archivo de firmas de reglas modular que permita a los desarrolladores personalizar y expandir los patrones de análisis según las políticas internas de la organización.
- Promover la cultura del desarrollo seguro (DevSecOps) mediante reportes automatizados en formatos PDF/JSON y la sincronización con repositorios mediante creación de tickets de seguridad.

3. Alcance del proyecto

Este sistema (Web, CLI y Extensión de IDE) estará dirigido en su versión piloto a Securify Software Solutions S.A.C. y sus equipos de desarrollo asociados. Incluirá funcionalidades de escaneo estático de archivos, identificación de dependencias inseguras (CVEs), verificación de cabeceras HTTP de URLs y la creación automática de issues en repositorios de GitHub. El sistema estará disponible para ejecución local (desarrollo interactivo) y acceso remoto a través de APIs de red, siendo compatible con el editor VS Code y los principales navegadores web.



4. Viabilidad del Sistema

El sistema cumple con los requisitos en términos de tecnología, recursos y costos. Tecnologías como Python y el framework FastAPI para el desarrollo backend, y una máquina virtual de Debian en la nube para el hosting y despliegue del servidor de aplicación y la base de datos MySQL, han sido verificadas en el estudio de viabilidad, y se consideran opciones viables y adecuadas para garantizar un desarrollo escalable y seguro. El equipo técnico cuenta con la experiencia necesaria para diseñar e implementar el sistema, asegurando que cumpla con los objetivos de calidad de software y funcionales propuestos. Además, el análisis financiero reveló un VAN favorable y una TIR alta, lo que confirma la viabilidad económica del proyecto y su sostenibilidad a largo plazo. La infraestructura en la nube (máquina virtual Debian) permitirá una escalabilidad fluida del sistema, mientras que el cumplimiento de las leyes locales y estándares internacionales de seguridad garantizará la protección de los datos de los usuarios, proporcionando confianza tanto a los desarrolladores como a los administradores en el uso de la plataforma.

5. Información obtenida del Levantamiento de Información

Se realizaron entrevistas con ingenieros de software líderes, analistas de QA y arquitectos de seguridad de Securify Software Solutions S.A.C. como parte del proceso de recopilación de información. Además, se aplicaron encuestas a los desarrolladores del equipo para comprender sus necesidades, expectativas y desafíos al momento de implementar pautas de desarrollo seguro bajo el estándar OWASP Top 10.

- Principales Hallazgos

- Los administradores y líderes de proyecto necesitan una herramienta tecnológica que les permita centralizar las auditorías, generar reportes automáticos y gestionar de forma eficiente los tokens y accesos del sistema.

- Los desarrolladores prefieren un sistema interactivo e integrado en el IDE (VS Code) que ofrezca retroalimentación en tiempo real (diagnósticos y subrayados) y sugerencias prácticas de remediación sin interrumpir su flujo de trabajo.

- Existe una falta de integración entre los diferentes métodos de análisis estático (SAST) y los repositorios remotos, lo que dificulta el seguimiento y registro centralizado de las vulnerabilidades.

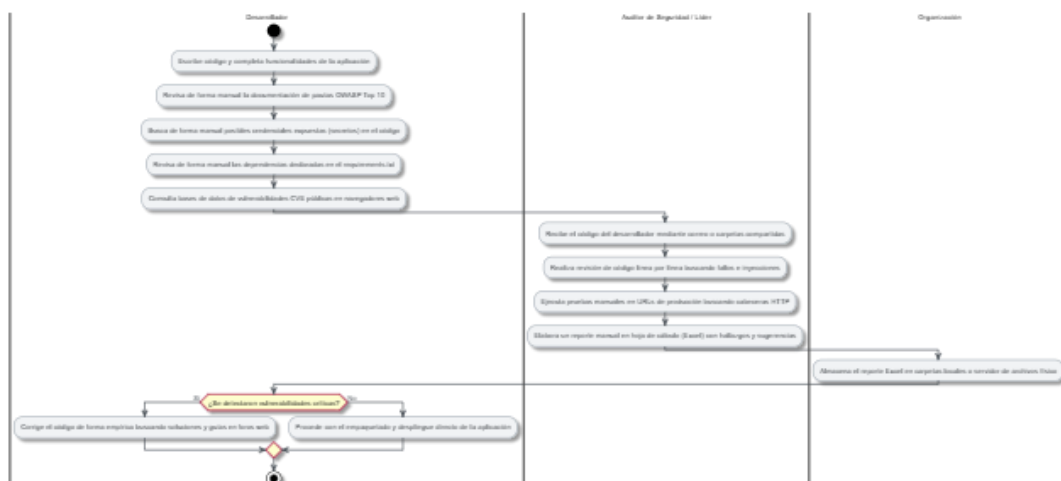
- Se identificó una creciente necesidad de automatizar la detección de dependencias vulnerables (CVEs), lo que resalta la importancia de incorporar escaneos automáticos de bibliotecas externas y la sincronización con issues de GitHub.

III. Análisis de Procesos

a) Diagrama del Proceso Actual – Diagrama de actividades

- Proceso Manual de Enseñanza y Aprendizaje

Figura 2: Proceso manual de enseñanza y aprendizaje

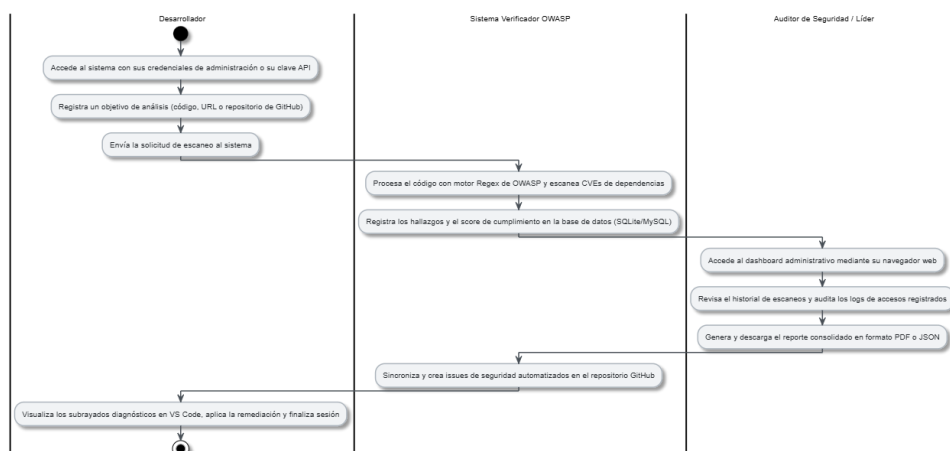


Fuente: Elaboración propia.

b) Diagrama del Proceso Propuesto – Diagrama de actividades Inicial:

- Proceso Propuesto para la Gestión de enseñanza y aprendizaje en el Sistema Web

Figura N°3: Proceso propuesto para la gestión de enseñanza y aprendizaje en el Sistema Web



Fuente: Elaboración propia.



IV. Especificación de Requerimientos de Software

a) Cuadro de Requerimientos funcionales Inicial:

Tabla N°1: Tabla de Requerimientos funcionales inicial

ID	Requerimiento Funcional	Descripción	Prioridad
RFI-01	Cargar código local	Permitir al usuario subir un archivo de código al servidor para su análisis.	Alta
RFI-02	Analizar patrones Regex	Buscar patrones de vulnerabilidades comunes utilizando expresiones regulares.	Alta
RFI-03	Escanear cabeceras URL	Realizar una petición a una dirección web y revisar cabeceras HTTP de respuesta.	Alta
RFI-04	Generar Reporte JSON	Retornar el detalle de hallazgos del análisis estructurado en formato JSON.	Media
RFI-05	Registrar en base de datos	Almacenar localmente el registro del escaneo realizado.	Media

Fuente: Elaboración Propia

b) Cuadro de Requerimientos No funcionales

Tabla N°2: Cuadro de Requerimientos no funcionales

ID	Requerimiento No Funcional	Descripción / Criterio de Medida
RNF01	Rendimiento	El tiempo de escaneo de código local debe ser menor a 5s, y de URLs menor a 15s.
RNF02	Seguridad de Acceso	Los endpoints de auditoría y generación de tokens requieren cabecera X-API-Key.
RNF03	Arquitectura Desacoplada	Separación estricta entre presentación, negocio (servicios) y acceso a datos.
RNF04	Portabilidad	Debe poder ejecutarse en Windows, Linux y macOS sin requerir complejas bases de datos pesadas (SQLite/MySQL dual).
RNF05	Usabilidad e Interfaz	El panel del IDE debe emplear estilos premium glassmorphism y ser totalmente adaptativo.
RNF06	Extensibilidad	El motor estático de escaneo Regex debe basarse en un archivo modular de firmas extensible.

Fuente: Elaboración Propia

c) Cuadro de Requerimientos funcionales Final

Tabla N°3: Cuadro de requerimientos funcionales final

ID	Nombre del Requerimiento	Descripción	Prioridad
RF01	Escanear objetivos	El sistema recibe código, archivo local, URL web o URL de repositorio de GitHub para su análisis.	Alta
RF02	Identificar dependencias	El sistema analiza dependencias declaradas y las coteja contra una lista predefinida de CVEs.	Alta
RF03	Generar y validar tokens de API	El sistema expone endpoints seguros y gestiona tokens de acceso para clientes externos.	Alta
RF04	Gestionar sesiones administrativas	El sistema administra las sesiones y tokens temporales de login/logout del administrador.	Alta
RF05	Gestionar usuarios	El administrador crea, edita, elimina usuarios y asigna roles.	Alta
RF06	Exportar reportes detallados	El sistema renderiza y exporta reportes interactivos en formato PDF y JSON.	Alta
RF07	Visualizar diagnósticos en IDE	La extensión de VS Code subraya líneas vulnerables directamente en el editor al guardar.	Alta
RF08	Guiar remediación en IDE	La extensión muestra remediaciones y sugerencias de código recomendadas según el framework.	Media
RF09	Monitorear y auditar accesos	El sistema registra el log de accesos de red (IP, ruta, usuario y fecha) de forma persistente.	Media

Fuente: Elaboración Propia

d) Reglas de Negocio:

Tabla N°4: Reglas de negocio

Nro.	Regla de Negocio	Descripción	REQ Asociado
RN01-1	Iniciar puntaje base de escaneo	Todo análisis de seguridad que comienza en el sistema debe inicializarse con un puntaje máximo de cumplimiento equivalente a 100 puntos (seguridad ideal).	RF01
RN01-2	Descontar penalizaciones por severidad	Por cada hallazgo de seguridad identificado en el código o URL, se restarán puntos del puntaje base según su gravedad: Alta (30 puntos), Media (15 puntos), Baja (5 puntos).	RF01
RN01-3	Limitar el puntaje mínimo de escaneo	El puntaje acumulado de un reporte de escaneo nunca podrá ser menor a 0 puntos, independientemente del número total de fallas encontradas.	RF01
RN01-4	Filtrar archivos mediante expresiones regulares	El escaneo estático solo debe procesar las líneas del archivo en base a los patrones regex estipulados para la lista oficial de reglas OWASP activas.	RF01
RN01-5	Validar protocolo de la URL	El sistema debe rechazar cualquier objetivo de tipo URL que no comience estrictamente con el protocolo seguro https:// o básico http://.	RF01
RN01-6	Bloquear conexiones a entornos locales	Para mitigar inyecciones SSRF, se debe cancelar el escaneo de cualquier URL que resuelva a localhost, IP local de bucle (127.0.0.1) o interfaces privadas.	RF01
RN01-7	Evaluar cabeceras de respuesta HTTP	El motor debe corroborar la presencia y el valor de cabeceras críticas como CSP, HSTS, X-Frame-Options y X-Content-Type-Options en la respuesta HTTP.	RF01
RN01-8	Descargar repositorios en formato comprimido	La descarga del código del repositorio remoto debe solicitarse a la API de GitHub en formato de archivo comprimido ZIP para ahorrar almacenamiento y tiempo.	RF01
RN01-9	Utilizar token de acceso de GitHub	Se debe inyectar el token del usuario en la cabecera HTTP de autorización si se requiere acceder a repositorios privados o eludir límites de tasa.	RF01
RN01-10	Extraer archivos de forma recursiva	Se debe descomprimir el ZIP en memoria y recorrer de forma recursiva sus directorios ignorando carpetas de dependencias (node_modules, .venv).	RF01
RN02-1	Comparar librerías contra base de datos CVE	Se mapean las dependencias detectadas contra la lista de identificadores CVE conocidos en el sistema para alertar si se usan componentes obsoletos.	RF02
RN03-1	Generar tokens únicos y seguros	Al registrar tokens para acceso a API se deben generar mediante algoritmos criptográficos aleatorios seguros asociados al usuario.	RF03
RN03-2	Validar credenciales en peticiones de API	Toda petición API a rutas /api requiere la cabecera X-API-Key para corroborar que el token existe, pertenece a un usuario y está activo.	RF03

RN04-1	Expirar sesiones administrativas	La cookie de sesión web de administración (admin_session) debe invalidarse automáticamente tras un límite máximo de 6 horas continuas.	RF04
RN04-2	Hashear contraseñas registradas	El sistema debe encriptar unidireccionalmente toda contraseña ingresada utilizando el algoritmo SHA-256 antes de su escritura final en base de datos.	RF04
RN05-1	Evitar duplicidad de usuarios	Se bloquea el registro de un nuevo usuario si su nombre ya coincide (sin distinguir mayúsculas de minúsculas) con uno existente en la base de datos.	RF05
RN05-2	Restringir eliminación de cuentas protegidas	Se impide el borrado físico del usuario maestro principal (admin) o del usuario que tiene la sesión activa de navegación web actual.	RF05
RN06-1	Formatear reportes en PDF descargables	El reporte exportable en PDF debe maquetar de forma estructurada los gráficos de severidad, puntajes y las guías de remediación de cada hallazgo.	RF06
RN06-2	Serializar reportes en formato JSON	La exportación a JSON debe estructurar únicamente los metadatos globales del escaneo y la lista detallada de hallazgos para integración externa.	RF06
RN07-1	Subrayar líneas de código vulnerables	El editor de VS Code debe destacar visualmente (colores de severidad rojo/amarillo) las líneas del código que contengan hallazgos de seguridad.	RF07
RN07-2	Habilitar y desactivar diagnósticos en el editor	La renderización de diagnósticos en el IDE debe mostrarse u ocultarse basándose en el flag de configuración showDiagnostics (owaspVerificator.showDiagnostics).	RF07
RN08-1	Detectar frameworks de desarrollo activos	El analizador de código debe escanear importaciones y patrones para descubrir el framework usado (FastAPI, Flask, Express, Django) y contextualizar la solución.	RF08
RN08-2	Adaptar remediación sugerida al contexto	El sistema entrega la solución adaptada con la sintaxis del framework específico, permitiendo un copiado de prompt rápido para la IA.	RF08
RN09-1	Registrar accesos en base de datos	Toda petición HTTP (excluyendo archivos estáticos) debe registrar en base de datos la IP, User-Agent, ruta y marca temporal mediante un middleware.	RF09
RN09-2	Depurar IPs repetidas en monitor	La vista de monitoreo de accesos del panel de control del dashboard debe filtrar duplicaciones y mostrar solo el acceso más reciente por IP única.	RF09

Fuente: Elaboración propia



V. Fase de Desarrollo

1. Perfiles de Usuario

Tabla N°5: Perfiles de usuario

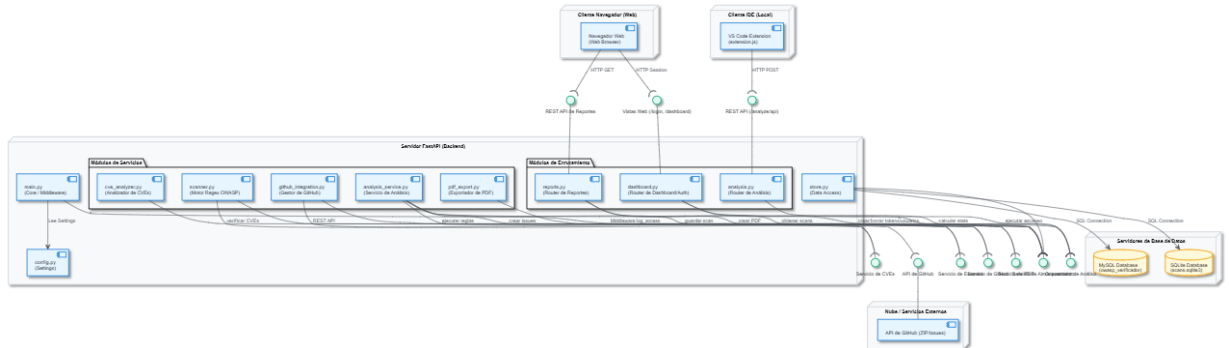
Perfil	Descripción	Acciones Clave	Beneficios
Administrador	Usuario con privilegios elevados encargado de la gestión general de la plataforma, control de accesos de red, administración de usuarios y auditoría global.	- Acceder al dashboard de administración web general. - Crear, gestionar y eliminar cualquier cuenta de usuario. - Generar y revocar tokens de API de cualquier usuario. - Exportar reportes de vulnerabilidades en PDF o JSON. - Visualizar estadísticas globales e historial de visitas por IP en tiempo real. - Habilitar o deshabilitar configuraciones globales del sistema.	- Obtiene visibilidad completa y centralizada de todas las auditorías realizadas en el sistema. - Controla de forma estricta los accesos y credenciales que interactúan con la plataforma. - Facilita la generación de informes consolidados de seguridad.
Usuario	Usuario general que utiliza la plataforma web o la extensión de VS Code para validar la seguridad de su propio código y recursos.	- Realizar análisis de código, URLs web o repositorios de GitHub. - Generar, copiar y utilizar sus propios tokens de acceso para la API REST. - Visualizar su historial personal de escaneos y estadísticas en el dashboard. - Consultar las advertencias visuales en el IDE (subrayados) al guardar código. - Acceder a guías de remediación adaptadas al framework de desarrollo con asistencia de IA.	- Recibe alertas de vulnerabilidades de manera inmediata durante la codificación. - Cuenta con un historial organizado de sus análisis personales. - Acelera la resolución de fallas OWASP gracias a la remediación contextualizada.

Fuente: Elaboración propia

2. Modelo Conceptual

a) Diagrama de Componentes:

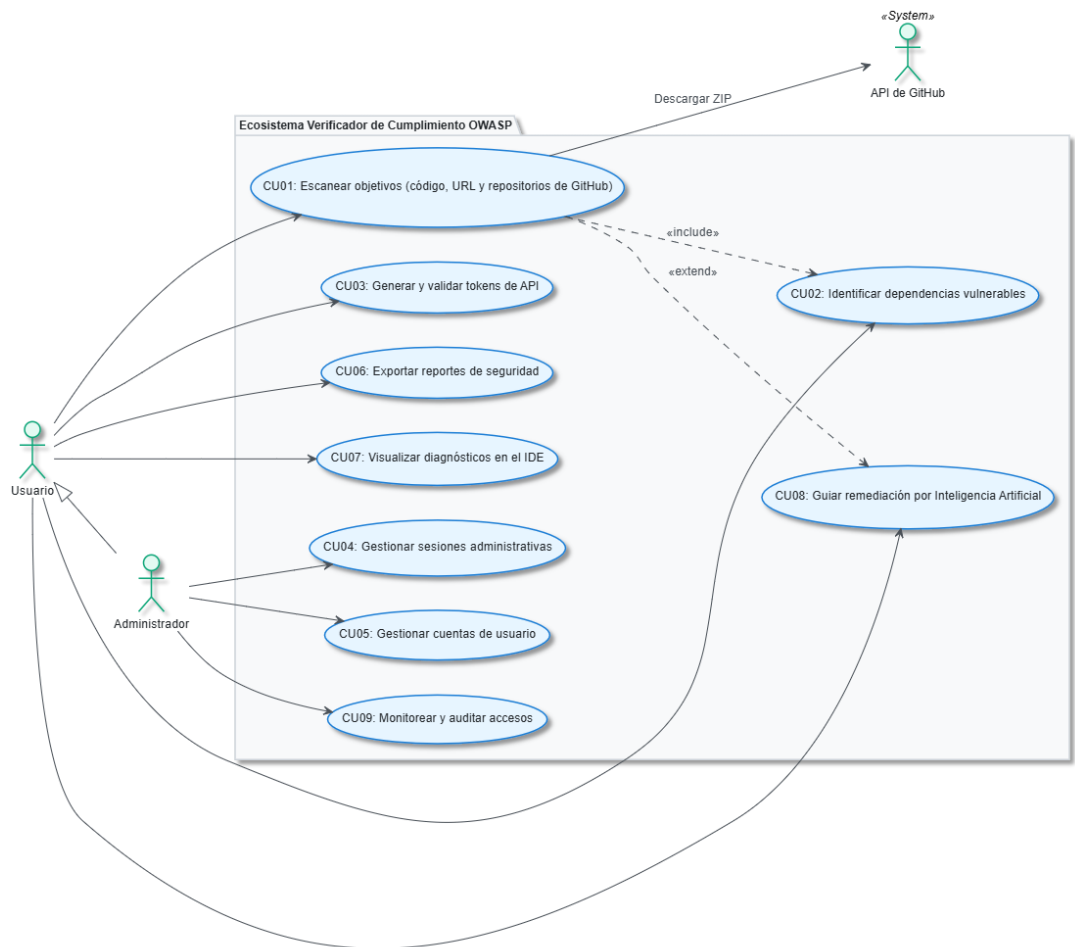
Figura N°4: [Diagrama de componentes](#)



Fuente: Elaboración propia

a) Diagrama de caso de uso general:

Figura N°5: Diagrama de [caso de uso general](#)



Fuente: Elaboración propia



b) Escenarios de Caso de Uso (narrativa)

Tabla N°6: Escenario-Escanear Objetivos

Escenario de caso de uso: Escanear objetivos (código, URL y repositorios de GitHub)	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario, API de GitHub (Actor Secundario)
Descripción	Permite al usuario realizar auditorías de seguridad sobre diferentes objetivos (archivos de código locales, texto pegado, URLs públicas de producción o repositorios remotos de GitHub) para verificar el cumplimiento de las directrices OWASP Top 10.
Precondiciones	El usuario debe encontrarse en el formulario de análisis del portal web o enviar una petición HTTP REST.
Postcondiciones	Se genera un reporte detallado con los hallazgos de seguridad detectados, se calcula el puntaje final de cumplimiento (de 0 a 100) y se guarda el historial del análisis en la base de datos (MySQL/SQLite).
Narrativa del Caso de Uso (Flujo Principal - Código Local por Archivo)	
Acción del Actor	Respuesta del Sistema
1. El usuario selecciona la opción "Archivo" en el formulario de análisis.	2. El sistema habilita el cargador de archivos locales del navegador.
3. El usuario selecciona su archivo de código local (`.py`, `.js`, etc.) y hace clic en "Analizar".	4. El sistema lee el contenido del archivo, valida que no esté vacío y ejecuta el motor estático aplicando expresiones regulares y buscando vulnerabilidades.
	5. El sistema calcula la puntuación final de cumplimiento (restando penalidades de 30, 15 y 5 puntos según severidad).
	6. El sistema persiste el reporte de escaneo en la base de datos y redirige a la vista detallada (`/reports/{scan_id}`).
Flujo Alternativo - FA1 (Cargar código mediante Texto Directo)	
Acción del Actor	Respuesta del Sistema
1. El usuario selecciona la opción "Texto" en el formulario.	2. El sistema oculta el cargador de archivos y habilita una caja de texto grande para entrada directa.

3. El usuario pega el fragmento de código en la caja de texto y hace clic en "Analizar".	4. El sistema lee el texto directo, valida que no esté vacío y continúa con el paso 4 del **Flujo Principal**.
Flujo Alternativo - FA2 (Analizar URL Web Pública)	
Acción del Actor	Respuesta del Sistema
1. El usuario selecciona la opción "URL" en el formulario de análisis.	2. El sistema habilita el campo de texto para ingresar direcciones web.
3. El usuario escribe la URL (ej. `https://example.com`) y hace clic en "Analizar".	4. El sistema valida el protocolo, resuelve el host y comprueba que la IP no sea local o privada (mitigando SSRF).
	5. El sistema realiza una petición HTTP GET externa y recupera las cabeceras del servidor.
	6. El sistema valida cabeceras críticas (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Server), calcula la puntuación (restando 15 pts por cabecera ausente) e inicia el paso 6 del **Flujo Principal**.
Flujo Alternativo - FA3 (Analizar Repositorio de GitHub)	
Acción del Actor	Respuesta del Sistema
1. El usuario selecciona la opción "GitHub Repo" en el formulario.	2. El sistema habilita los campos de URL de repositorio y Token de acceso opcional.
3. El usuario escribe la URL de GitHub (y opcionalmente su token) y hace clic en "Analizar".	4. El sistema solicita a la API de GitHub la descarga de la rama principal del repositorio en formato comprimido ZIP. Si es privado, inyecta el token en la cabecera `Authorization`.
	5. El sistema procesa el ZIP directamente en memoria, descomprime sus archivos y aplica el análisis estático recursivo sobre archivos legibles de código (excluyendo carpetas pesadas).
	6. El sistema acumula las penalidades del análisis, calcula el puntaje final y ejecuta el paso 6 del **Flujo Principal**.
Flujo de Excepciones - FE1 (Archivo o campo de análisis vacío)	
Acción del Actor	Respuesta del Sistema
1. El usuario hace clic en "Analizar" sin seleccionar un archivo, sin pegar texto o con campos vacíos.	2. El sistema intercepta el formulario, detiene la ejecución del análisis y muestra una alerta: "Por favor selecciona un archivo" o "target_value es requerido".
Flujo de Excepciones - FE2 (Falla en lectura de archivo local)	
Acción del Actor	Respuesta del Sistema

1. El usuario sube un archivo binario o corrupto.	2. El sistema intenta leer el texto, captura el fallo de decodificación de caracteres y retorna un error HTTP 400 señalando formato inválido.
Flujo de Excepciones - FE3 (URL sin protocolo o inválida)	
Acción del Actor	Respuesta del Sistema
1. El usuario ingresa una dirección de sitio web sin prefijo (ej. `example.com`).	2. El sistema detecta la ausencia del protocolo `http://` o `https://`, bloquea la ejecución y muestra la advertencia: "URL inválida".
Flujo de Excepciones - FE4 (SSRF detectado en URL)	
Acción del Actor	Respuesta del Sistema
1. El usuario ingresa una dirección que apunta a la red local del servidor (ej. `http://localhost` o `127.0.0.1`).	2. El sistema resuelve la IP, identifica el rango local/privado, detiene la petición de red y devuelve la alerta: "Acceso denegado a rangos IP privados".
Flujo de Excepciones - FE5 (Host web caído o inactivo)	
Acción del Actor	Respuesta del Sistema
1. El usuario ingresa un dominio inactivo que no responde.	2. El sistema intenta conectarse y, tras agotar el tiempo de espera (timeout), cancela la operation y notifica la falla de red.
Flujo de Excepciones - FE6 (Repositorio privado de GitHub inaccesible)	
Acción del Actor	Respuesta del Sistema
1. El usuario ingresa la URL de un repositorio privado sin proporcionar un token correcto.	2. El sistema recibe una respuesta HTTP 404/401 de la API de GitHub, aborta la descarga y avisa: "No se pudo acceder al repositorio. Verifique que sea público o que el token sea correcto".
Flujo de Excepciones - FE7 (Cuota de API de GitHub agotada)	
Acción del Actor	Respuesta del Sistema
1. El usuario realiza múltiples descargas consecutivas anónimas de GitHub.	2. El sistema intercepta el código de error HTTP 403 (Rate Limit Exceeded) de GitHub y notifica al usuario la necesidad de iniciar sesión y configurar un Token personal.

Fuente: Elaboración propia

Tabla N°7: Escenario de caso de uso- Identificar dependencias vulnerables

Escenario de caso de uso: Identificar dependencias vulnerables	
Tipo	Obligatorio (Incluido en CU01)
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al sistema revisar las dependencias importadas en el código de desarrollo o listadas en un archivo de requerimientos (`requirements.txt`) para comprobar si tienen vulnerabilidades conocidas (CVEs).
Precondiciones	El usuario debe haber iniciado un escaneo de código (CU01) que contenga importaciones de librerías o un archivo `requirements.txt`.
Postcondiciones	Se genera un listado de dependencias vulnerables identificadas que se asocian como hallazgos de seguridad en el reporte generado.
Narrativa del Caso de Uso (Flujo Principal - Detección de CVEs)	
Acción del Actor	Respuesta del Sistema
1. El usuario inicia el escaneo del código fuente o repositorio.	2. El sistema detecta la presencia de importaciones de librerías comunes (como Flask, Django, requests) o la existencia de un archivo `requirements.txt`.
	3. El sistema analiza el nombre y la versión de cada dependencia listada.
	4. El sistema contrasta las versiones de las librerías contra su base de datos local de vulnerabilidades conocidas (CVEs).
	5. Si coincide con una versión vulnerable registrada, el sistema genera un hallazgo de severidad media/alta detallando la vulnerabilidad e indicando la remediación (actualizar librería) en el reporte final.
Flujo Alternativo - FA1 (Análisis automático en IDE al guardar archivo)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador guarda un archivo de declaración de dependencias (`requirements.txt`) en el IDE de VS Code.	2. La extensión envía el archivo al backend de manera asíncrona. El sistema ejecuta la validación de CVEs en la base de datos de inmediato y retorna la lista de dependencias comprometidas para ser subrayadas en el IDE.
Flujo de Excepciones - FE1 (Archivo de dependencias vacío o ilegible)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador adjunta o escanea un archivo `requirements.txt` corrupto o con formato no estándar.	2. El sistema intenta procesar el archivo, identifica que la estructura de líneas no cumple con el formato estándar `paquete==version`, omite el análisis de CVEs para las líneas mal estructuradas y continúa con el escaneo del resto del código.

Fuente: Elaboración propia



Tabla N°8: Escenario de caso de uso- Generar y validar tokens de API

Escenario de caso de uso: Generar y validar tokens de API	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al usuario generar nuevos tokens de autenticación API para integraciones con su entorno local (IDE) o herramientas externas, y al sistema validar dichas credenciales en cada petición REST.
Precondiciones	El usuario debe estar logueado y encontrarse en el dashboard del sistema, o realizar una petición REST con una cabecera X-API-Key.
Postcondiciones	Se registra un nuevo token único en estado activo en la base de datos, o se valida exitosamente el token en una solicitud entrante.
Narrativa del Caso de Uso (Flujo Principal - Generar Token)	
Acción del Actor	Respuesta del Sistema
1. El usuario navega a la sección de "Tokens de API" en el dashboard.	2. El sistema muestra la lista de sus tokens y la opción de generar uno nuevo.
3. El usuario hace clic en el botón "Generar nuevo token".	4. El sistema genera una cadena aleatoria única y la asocia a la cuenta del usuario actual.
	5. El sistema registra el token, fecha de creación y estado activo en la base de datos (MySQL/SQLite).
	6. El sistema actualiza la lista y muestra en pantalla la clave generada para que el usuario pueda copiarla.
Flujo Alternativo - FA1 (Validar Token en peticiones de API)	
Acción del Actor	Respuesta del Sistema
1. Un cliente externo (o la extensión del IDE) envía una solicitud HTTP a la API REST del backend incluyendo la cabecera `X-API-Key` con el valor del token.	2. El sistema intercepta la cabecera de la petición, consulta la base de datos de tokens activos y valida que sea correcto.
	3. Si el token es válido y está activo, el sistema autoriza la operación y devuelve la respuesta correspondiente en formato JSON.
Flujo de Excepciones - FE1 (Token inválido o expirado)	
Acción del Actor	Respuesta del Sistema
1. Un cliente de integración externa envía una solicitud con una cabecera `X-API-Key` vacía, errónea o desactivada.	2. El sistema intercepta el token, realiza la búsqueda en base de datos y al no encontrar coincidencias válidas, rechaza la solicitud retornando un error HTTP 401 (No autorizado / Token inválido).

Fuente: Elaboración propia



Tabla N°9: Escenario de caso de uso- Gestionar sesiones administrativas

Escenario de caso de uso: Gestionar sesiones administrativas	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Administrador
Descripción	Permite controlar de forma segura el inicio de sesión, el cierre y la expiración automática del token de sesión web para el perfil de administración.
Precondiciones	El usuario debe tener una cuenta de administrador registrada en el sistema y encontrarse en la página de login.
Postcondiciones	Se establece una cookie segura `admin_session` en el navegador y se registra el ID de sesión activo en la base de datos.
Narrativa del Caso de Uso (Flujo Principal - Iniciar Sesión)	
Acción del Actor	Respuesta del Sistema
1. El administrador ingresa a la dirección `/login` en el navegador.	2. El sistema muestra la interfaz de inicio de sesión con campos para Nombre de Usuario y Contraseña.
3. El administrador rellena sus credenciales y presiona "Iniciar Sesión".	4. El sistema captura la contraseña, la hashea utilizando SHA-256 y verifica las credenciales contra la base de datos.
	5. Si son correctas, el sistema genera un ID de sesión criptográfico, lo guarda en la tabla `admin_sessions` especificando expiración en 6 horas y establece la cookie `admin_session` (con flags HttpOnly y SameSite=lax) redirigiendo al dashboard (`/dashboard`).
Flujo Alternativo - FA1 (Cerrar Sesión - Logout)	
Acción del Actor	Respuesta del Sistema
1. El administrador hace clic en el enlace "Cerrar sesión" en el menú de navegación superior.	2. El sistema captura el ID de sesión de la cookie, busca el registro en la base de datos y lo revoca/elimina.
	3. El sistema destruye la cookie de sesión en el cliente y redirige a la página principal del portal.
Flujo de Excepciones - FE1 (Credenciales de acceso incorrectas)	
Acción del Actor	Respuesta del Sistema
1. El administrador ingresa una contraseña o usuario incorrecto y presiona "Iniciar Sesión".	2. El sistema rechaza las credenciales hasheadas, detiene el flujo de creación de sesión y muestra un mensaje de error: "Usuario o contraseña incorrectos" con código de estado HTTP 401.
Flujo de Excepciones - FE2 (Sesión expirada tras inactividad)	
Acción del Actor	Respuesta del Sistema
1. El administrador intenta navegar en el panel después de 6 horas continuas de su inicio de sesión inicial.	2. El middleware intercepta la petición, comprueba la marca temporal en la base de datos, determina que la sesión ha expirado, destruye la cookie en el cliente y redirige automáticamente al usuario al `/login`.

Fuente: Elaboración propia



Tabla N°10: Escenario de caso de uso- Gestionar cuentas de usuario

Escenario de caso de uso: Gestionar cuentas de usuario	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Administrador
Descripción	Permite al administrador crear nuevas cuentas de usuarios (con rol de usuario común o administrador) y eliminar cuentas del sistema.
Precondiciones	El administrador debe haber iniciado sesión en el sistema y encontrarse en el panel de control de usuarios.
Postcondiciones	Las cuentas de usuario son creadas o eliminadas en la base de datos sqlite del sistema de forma persistente.
Narrativa del Caso de Uso (Flujo Principal - Crear/Eliminar por el Admin)	
Acción del Actor	Respuesta del Sistema
1. El administrador accede a la pestaña de "Usuarios" en el Dashboard.	2. El sistema muestra la lista de usuarios activos y el formulario de registro de nuevas cuentas.
3. El administrador rellena los campos (Nombre de Usuario, Email y Contraseña) y presiona "Crear".	4. El sistema valida que los campos requeridos no estén vacíos y que el nombre de usuario sea único.
	5. El sistema hashea la contraseña proporcionada utilizando el algoritmo criptográfico SHA-256 y guarda la cuenta con rol `admin` en la tabla `users`.
6. El administrador pulsa el botón "Eliminar" asociado al usuario que desea borrar de la lista.	7. El sistema valida que el nombre no corresponda al administrador maestro (`admin`) ni al usuario logueado en la sesión activa. Si pasa, ejecuta el borrado SQL y recarga la interfaz.
Flujo Alternativo - FA1 (Crear usuario general desde el formulario de registro público)	
Acción del Actor	Respuesta del Sistema
1. Un usuario visitante ingresa a la ruta `/register` de la web en lugar de hacerlo desde el dashboard administrativo.	2. Muestra la pantalla pública de registro de usuarios generales.
3. El visitante rellena el formulario de registro con sus datos y presiona "Registrarse".	4. El sistema valida los campos, hashea la contraseña, guarda la cuenta con rol general de tipo `user` por defecto en la base de datos, y redirige a la pantalla de login.
Flujo de Excepciones - FE1 (Intento de registrar un nombre de usuario duplicado)	
Acción del Actor	Respuesta del Sistema
1. El administrador (o visitante) intenta crear un usuario cuyo nombre ya está registrado en la base de datos.	2. El sistema detecta la colisión del nombre en la base de datos, cancela la inserción SQL, detiene el proceso y muestra una alerta: "El usuario ya está registrado".

Fuente: Elaboración propia

Tabla N°11: Escenario de caso de uso- Exportar reportes de seguridad

Escenario de caso de uso: Exportar reportes de seguridad	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al usuario descargar los resultados del análisis de seguridad en un archivo PDF estructurado (para documentación) o un archivo JSON (para integraciones).
Precondiciones	El reporte solicitado debe estar registrado en el historial de base de datos del sistema.
Postcondiciones	Se envía un flujo de descarga del archivo solicitado (`reporte-seguridad-{id}.pdf` o `.json`) al navegador del usuario.
Narrativa del Caso de Uso (Flujo Principal - Exportar a PDF)	
Acción del Actor	Respuesta del Sistema
1. El usuario hace clic en el botón "Exportar PDF" en la cabecera de la vista de un reporte detallado.	2. El sistema lee el ID del escaneo solicitado desde la ruta del navegador.
	3. El sistema realiza la consulta a la base de datos para obtener los metadatos y hallazgos vinculados al escaneo.
	4. El sistema invoca al servicio de generación de PDF para crear el documento con tablas y esquemas de color de severidad.
	5. El sistema añade la cabecera `Content-Disposition: attachment` y retorna el archivo como un flujo descargable (`StreamingResponse`).
Flujo Alternativo - FA1 (Descargar el reporte en formato JSON)	
Acción del Actor	Respuesta del Sistema
1. El usuario hace clic en el botón "Exportar JSON" en la vista del reporte.	2. El sistema intercepta la petición, lee el ID y recupera los datos del escaneo desde la base de datos.
	3. Mapea y serializa los datos del escaneo (`id`, `target_type`, `score`, `findings`) a una cadena JSON estructurada.
	4. El sistema añade la cabecera `Content-Disposition: attachment` y retorna el archivo al navegador en formato JSON legible (`JSONResponse`).
Flujo de Excepciones - FE1 (Reporte no registrado en base de datos)	
Acción del Actor	Respuesta del Sistema
1. El usuario modifica la URL solicitando exportar un ID de escaneo inexistente (ej. `/reports/99999/export-pdf`).	2. El sistema realiza la búsqueda en base de datos, no encuentra ningún registro, aborta el flujo de generación del archivo y retorna un error HTTP 404 (Scan no encontrado).

Fuente: Elaboración propia



Tabla N°12: Escenario de caso de uso- Visualizar diagnósticos en el IDE

Escenario de caso de uso: Visualizar diagnósticos en el IDE	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al desarrollador observar los fallos de seguridad detectados de manera directa sobre las líneas de su código en tiempo real en VS Code.
Precondiciones	La extensión debe estar instalada en VS Code, configurada con un token de API válido y el servidor backend debe estar en ejecución.
Postcondiciones	El editor de VS Code dibuja subrayados coloreados (rojo para severidades altas, amarillo para medias) sobre las líneas vulnerables detectadas.
Narrativa del Caso de Uso (Flujo Principal - Subrayados en el Editor)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador modifica un archivo de código local y lo guarda.	2. La extensión de VS Code detecta el evento de guardado en segundo plano de manera asíncrona.
	3. La extensión envía el contenido del código y el token al servidor backend.
	4. El servidor procesa las reglas OWASP, identifica fallas y responde con los hallazgos en JSON.
	5. La extensión recibe el JSON, borra marcas antiguas y dibuja subrayados de color (rojo/amarillo) en las líneas afectadas del editor de código.
Flujo Alternativo - FA1 (Visualizar hallazgos en la barra lateral - Dashboard)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador abre la barra lateral de la extensión en lugar de mirar los subrayados.	2. Abre un panel Webview interactivo que agrupa y lista todas las vulnerabilidades detectadas del proyecto.
3. El desarrollador hace clic sobre un archivo listado con vulnerabilidades en el panel.	4. El sistema abre el archivo enfocando el cursor en la línea del hallazgo y muestra la sugerencia de remediación.
Flujo de Excepciones - FE1 (Diagnósticos desactivados por configuración)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador tiene definida la configuración `owaspVerificator.showDiagnostics` en falso (`false`).	2. El sistema solicita y procesa el análisis con el backend de forma oculta, pero restringe la renderización de subrayados de error en las pestañas de edición del código.

Fuente: Elaboración propia



Tabla N°13: Escenario de caso de uso- Guiar remediación por Inteligencia Artificial

Escenario de caso de uso: Guiar remediación por Inteligencia Artificial	
Tipo	Opcional (Extendida de CU01)
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al desarrollador ver soluciones de remediación contextualizadas e integrarlas de forma directa en herramientas de IA (Copilot/Gemini).
Precondiciones	Se debe haber ejecutado un análisis y tener abiertos los resultados (reporte web o panel del IDE).
Postcondiciones	El sistema muestra los pasos de remediación ajustados y entrega un prompt de ingeniería listo para copiar en el portapapeles.
Narrativa del Caso de Uso (Flujo Principal - Remediación desde la Web)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador hace clic sobre un hallazgo listado en el reporte web detallado.	2. Muestra en pantalla el bloque de código que causó la penalidad.
3. El desarrollador presiona el botón "Ver Remediación" o "Generar Prompt de IA".	4. Ejecuta el detector de importaciones del archivo analizado buscando frameworks conocidos (FastAPI, Flask, Express, Django).
	5. Recupera la plantilla de remediación OWASP correspondiente para ese framework y compone un prompt estructurado de código.
	6. Renderiza los pasos de solución en pantalla y copia automáticamente el prompt generado en el portapapeles.
Flujo Alternativo - FA1 (Solicitar remediación desde la bombilla Quick Fix de VS Code)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador coloca el cursor sobre la línea subrayada en VS Code y abre la bombilla (Quick Fix / `Ctrl+.`).	2. El sistema despliega el menú de acciones rápidas de la extensión.
3. El desarrollador selecciona la opción "OWASP: Generar Prompt de Solución".	4. El sistema ejecuta la consulta de remediación adaptada de forma local y coloca el prompt formateado en el portapapeles del editor para usar con Copilot o Gemini.
Flujo de Excepciones - FE1 (Imposibilidad de identificar el framework - Remediación Genérica)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador solicita la remediación sobre un código sin importaciones explícitas de frameworks soportados.	2. Detecta que no puede asociar un framework web, utiliza la plantilla de remediación estándar del lenguaje y compone el prompt usando la sintaxis genérica por defecto.

Fuente: Elaboración propia

Tabla N°14: Escenario de caso de uso- Monitorear y auditar accesos

Escenario de caso de uso: Monitorear y auditar accesos	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Administrador
Descripción	Guarda un registro cronológico de peticiones HTTP en el servidor backend para auditoría e inspección de red, mostrando vistas detalladas en el panel administrativo.
Precondiciones	El administrador debe haber iniciado sesión en el sistema y encontrarse en el dashboard web.
Postcondiciones	Se listan los registros de auditoría de red depurados o secuenciales en pantalla.
Narrativa del Caso de Uso (Flujo Principal - Monitoreo por IP única)	
Acción del Actor	Respuesta del Sistema
1. El administrador hace clic en la pestaña "Monitoreo" del panel lateral administrativo.	2. El sistema consulta las marcas temporales e IPs almacenadas en la tabla `access_logs` de la base de datos (MySQL/SQLite).
	3. El sistema recorre los registros en orden descendente y aplica un filtro para dedupilar las direcciones IP repetidas, conservando solo la visita más reciente de cada IP única.
	4. El sistema renderiza la tabla de monitoreo mostrando: Dirección IP, Ruta del Recurso, User-Agent, Fecha y Hora del último acceso.
Flujo Alternativo - FA1 (Visualizar listado secuencial completo)	
Acción del Actor	Respuesta del Sistema
1. El administrador navega a la sección inferior de "Accesos Recientes" en la pantalla principal del Dashboard.	2. El sistema consulta las últimas 100 peticiones registradas de forma cronológica sin aplicar filtros de duplicidad.
	3. El sistema carga y muestra secuencialmente la lista completa en una tabla compacta para auditoría rápida de ráfagas de red.
Flujo de Excepciones - FE1 (Pérdida de privilegios o sesión)	
Acción del Actor	Respuesta del Sistema
1. El administrador intenta ingresar a `/monitoring` con la sesión web expirada o alterando su rol.	2. El sistema intercepta la petición en los controladores de ruta de FastAPI, valida que no posee permisos válidos de sesión, bloquea el acceso a la base de datos y redirige a la pantalla de login `/login`.

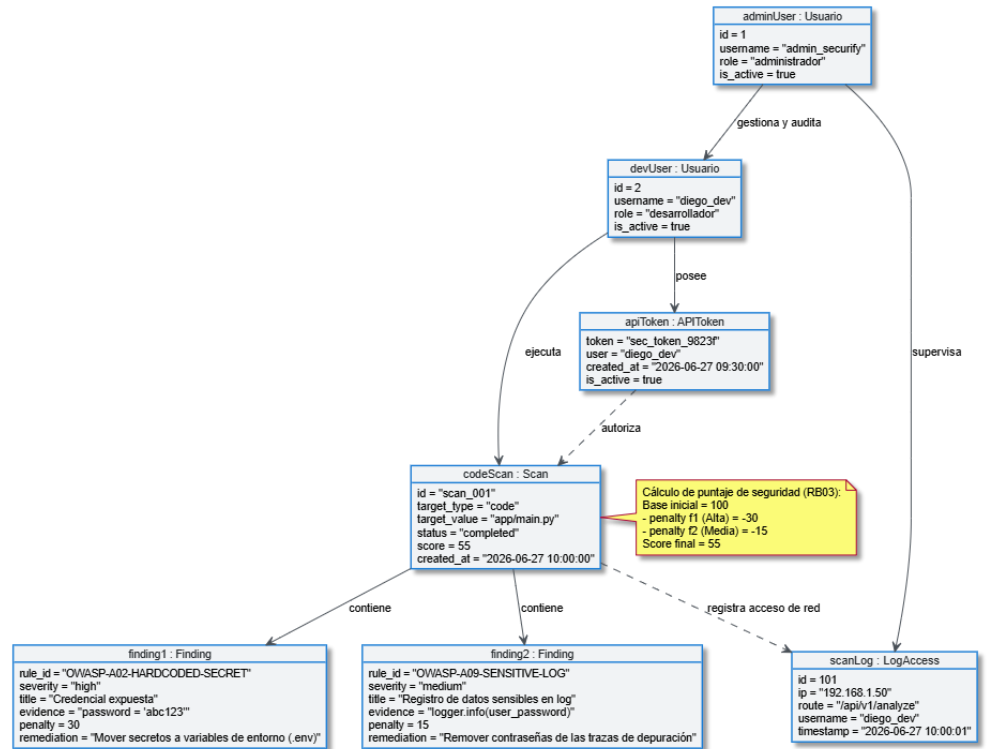
Fuente: Elaboración propia

3. Modelo Lógico

a) Análisis de Objetos

Diagrama de análisis de objetos

Figura N°6: Diagrama de análisis de objetos

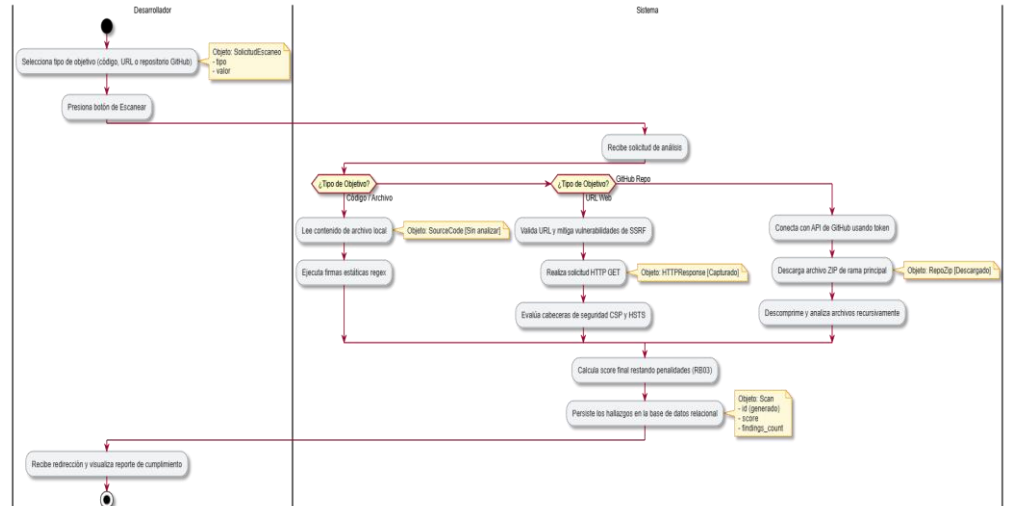


Fuente: Elaboración propia

b) Diagrama de actividades con objetos

a. Escanear Objetivos

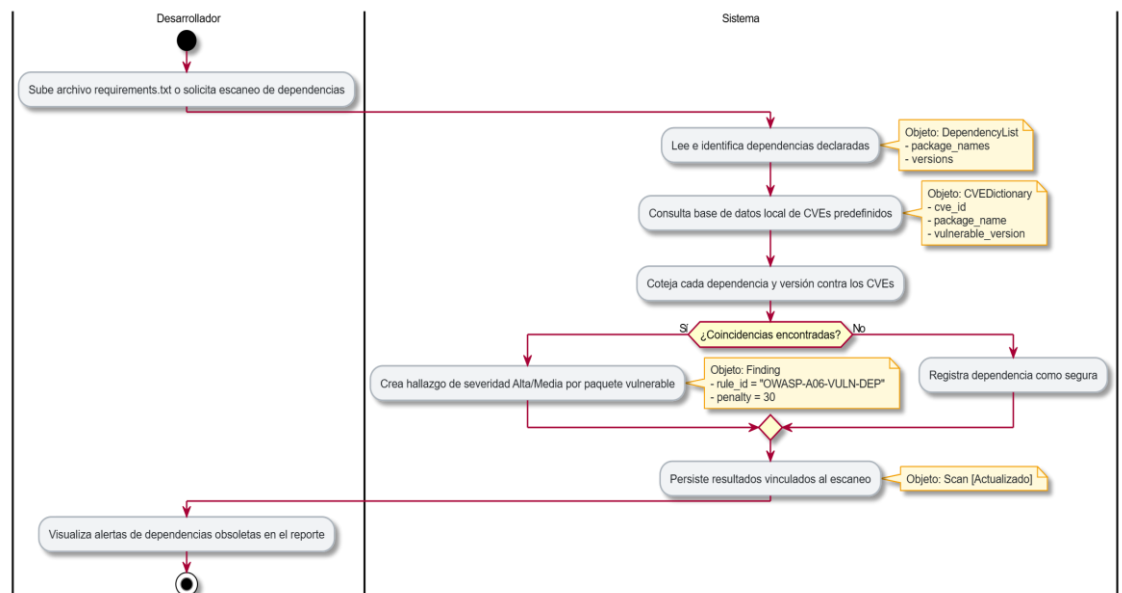
Figura N°7: Diagrama de actividades con objetos - Escanear objetivos



Fuente: Elaboración Propia

b. Identificar dependencias

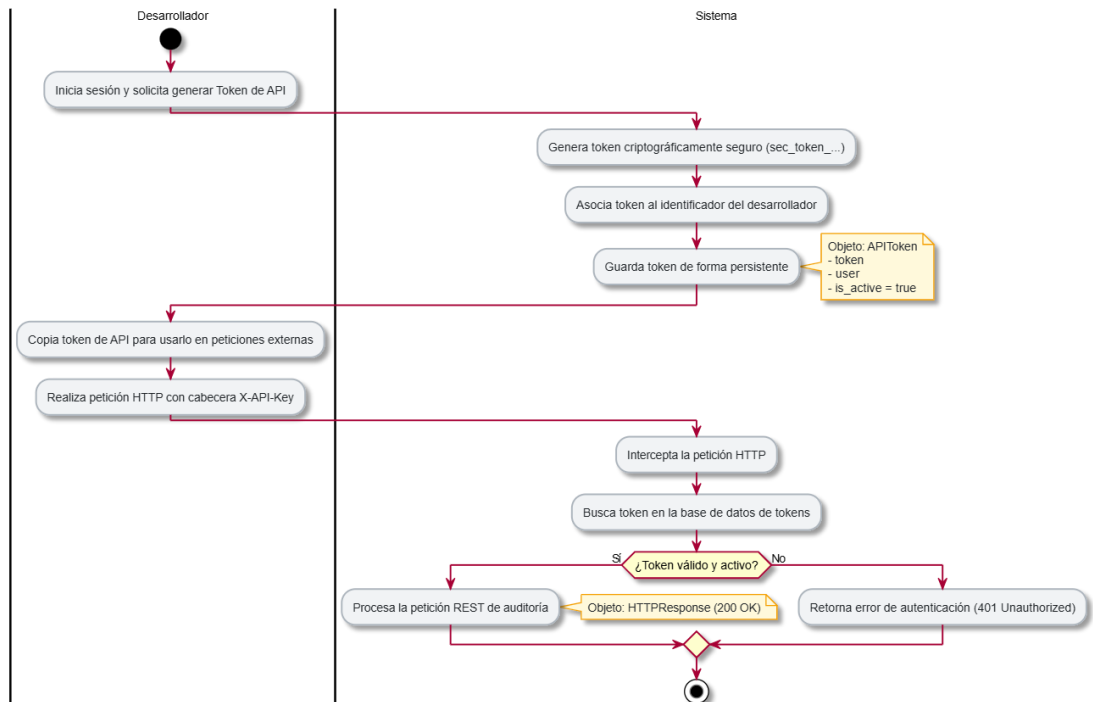
Figura N°8: Diagrama de actividades con objetos - Identificar dependencias



Fuente: Elaboración Propia

c. Generar y validar tokens de API

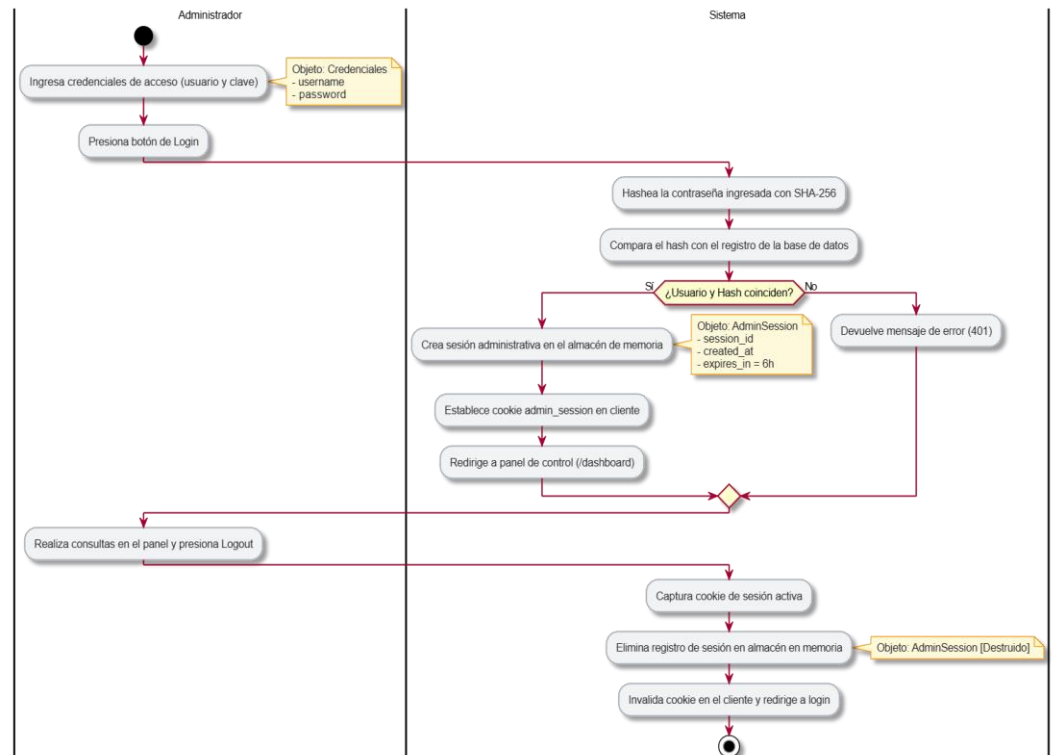
Figura N°9: Diagrama de actividades con objetos - Generar y validar tokens de API



Fuente: Elaboración propia

d. Gestionar sesiones administrativas

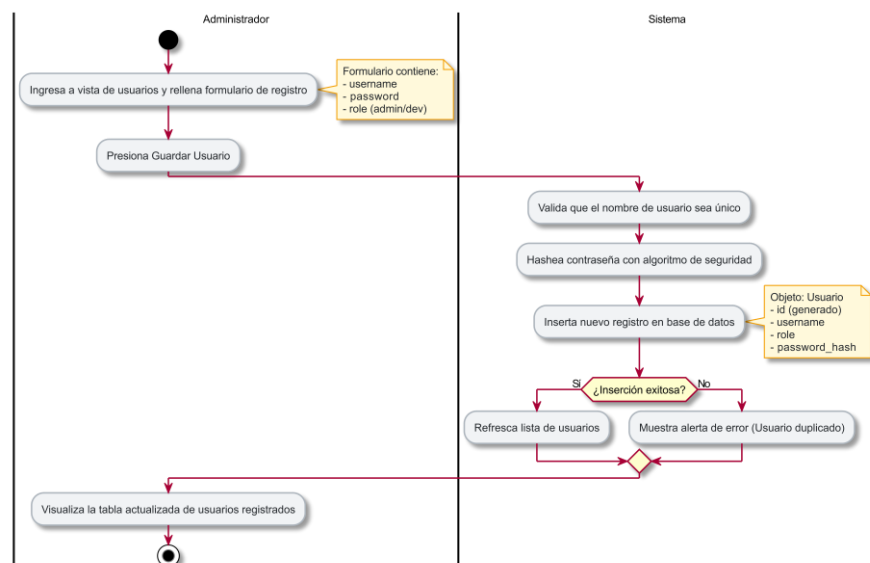
Figura N°10: Diagrama de actividades con objetos - Gestionar sesiones administrativas



Fuente: Elaboración propia

e. Gestionar usuarios

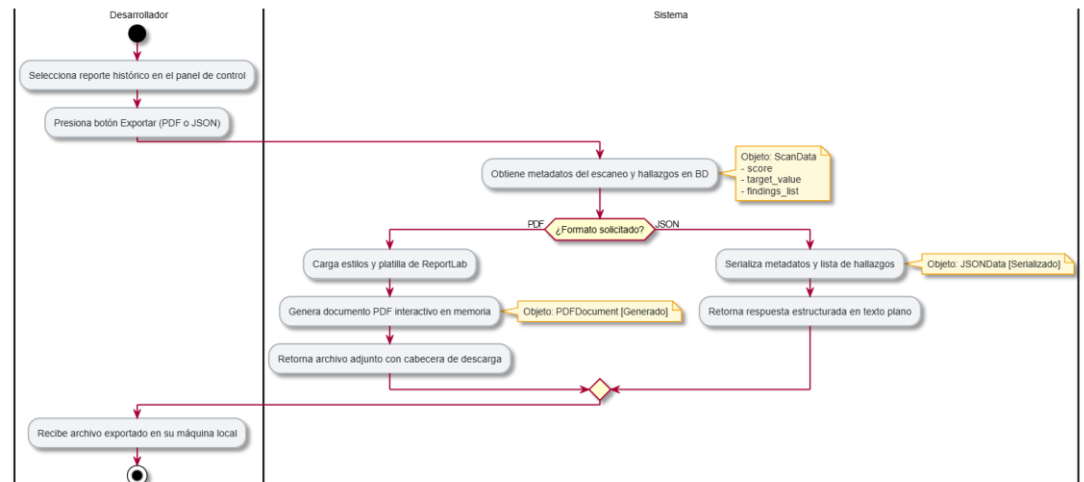
Figura N°11: Diagrama de actividades con objetos - Gestionar usuarios



Fuente: Elaboración propia

f. Exportar reportes detallados

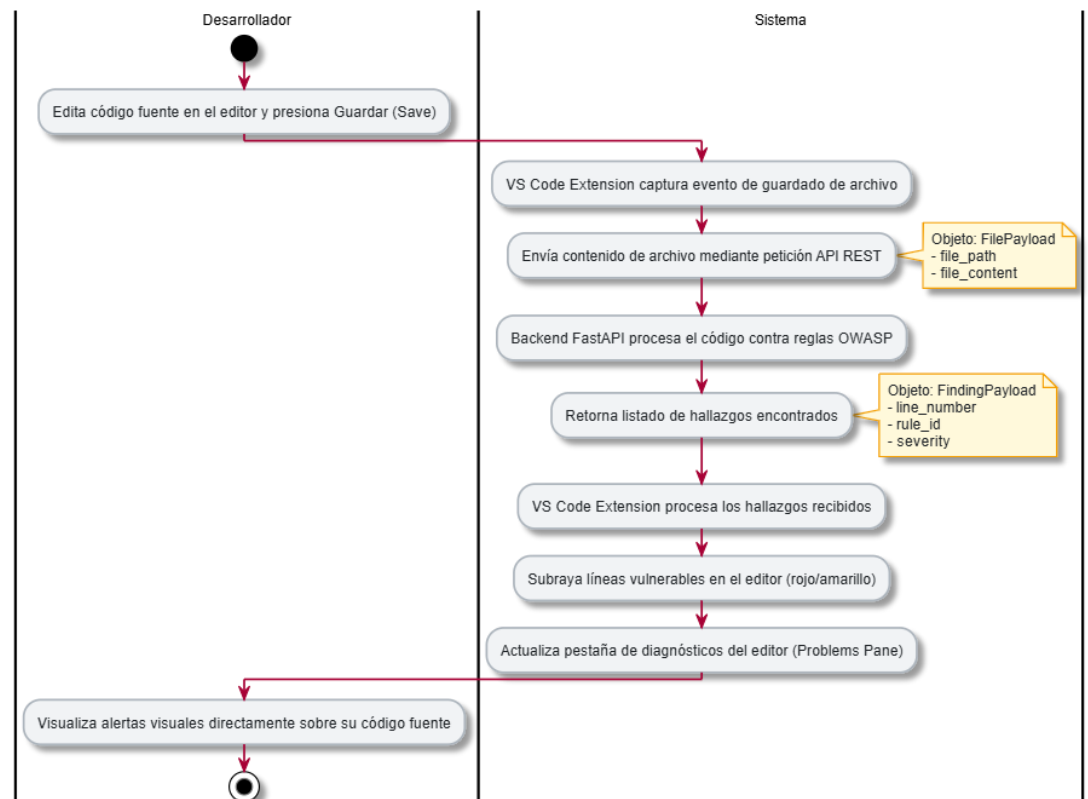
Figura N°12: Diagrama de actividades con objetos - Exportar reportes detallados



Fuente: Elaboración propia

g. Visualizar diagnósticos en IDE

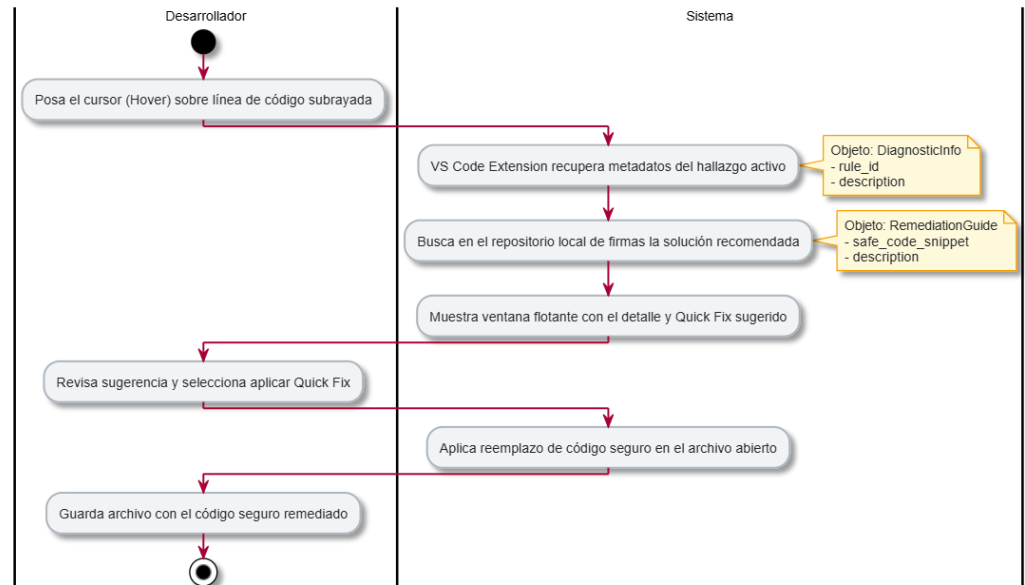
Figura N°13: Diagrama de actividades con objetos - Visualizar diagnósticos en IDE



Fuente: Elaboración propia

h. Guiar remediación por Inteligencia Artificial

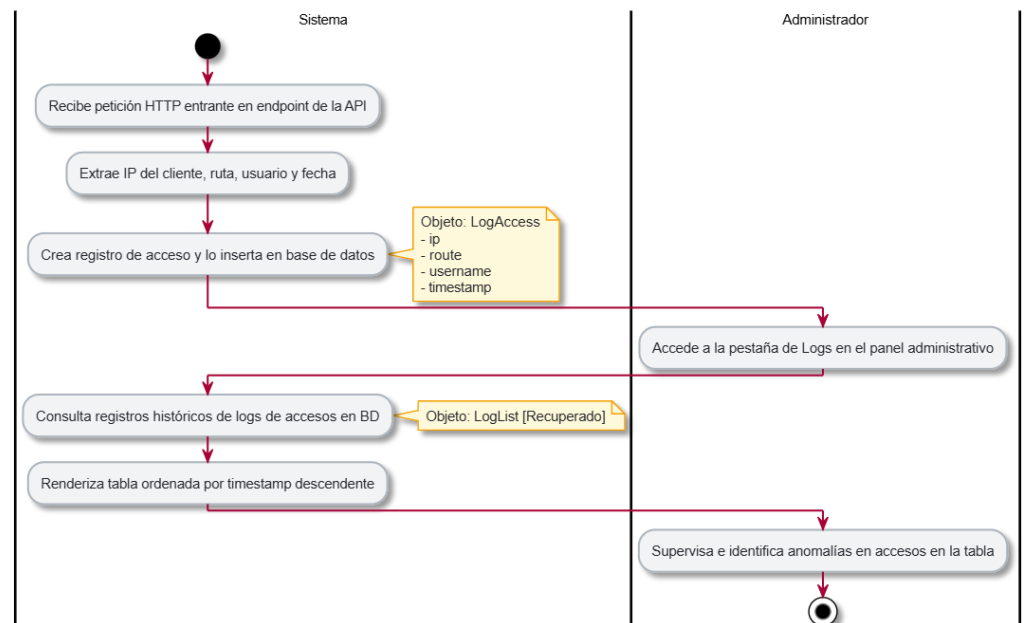
Figura N°14: Diagrama de actividades con objetos - Guiar remediación por Inteligencia Artificial



Fuente: Elaboración propia

i. Monitorear y auditar accesos

Figura N°15: Diagrama de actividades con objetos - Monitorear y auditar accesos

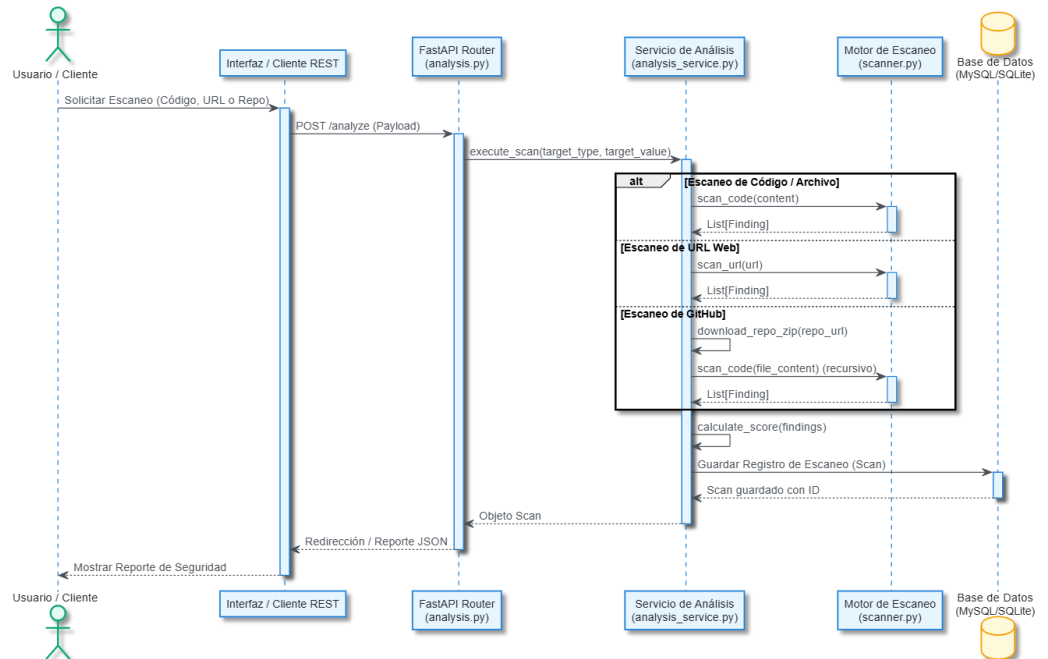


Fuente: Elaboración propia

c) Diagrama de Secuencia

a. Diagrama de secuencia- Escanear objetivos

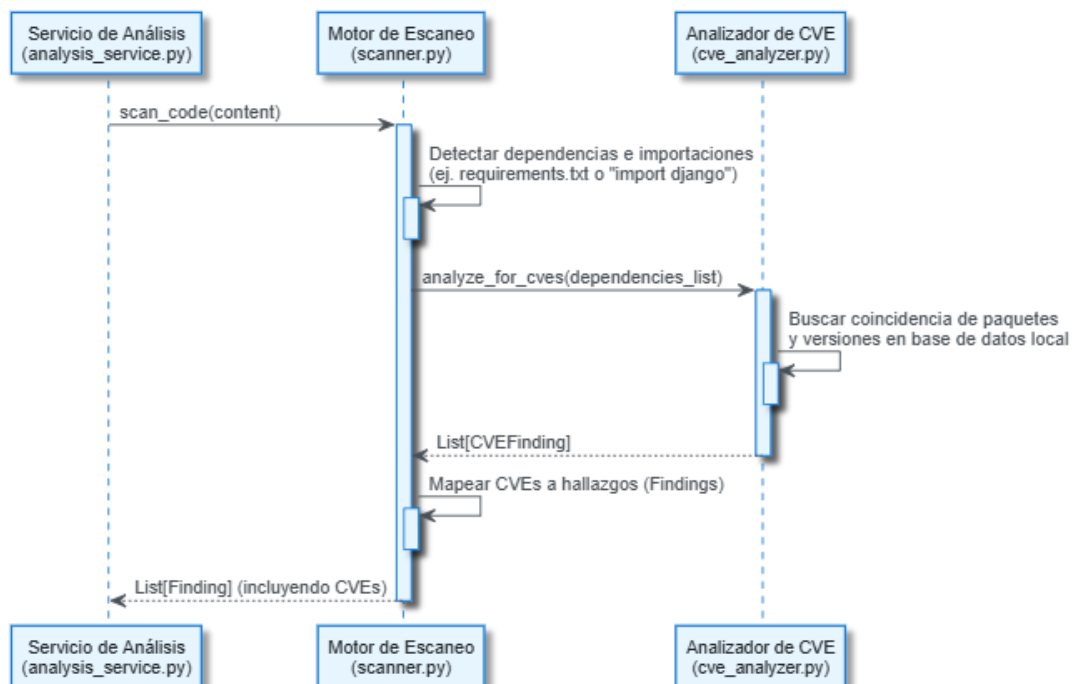
Figura N°21: Diagrama de secuencia - Escanear objetivos



Fuente: Elaboración Propia

b. Diagrama de secuencia- Identificar dependencias

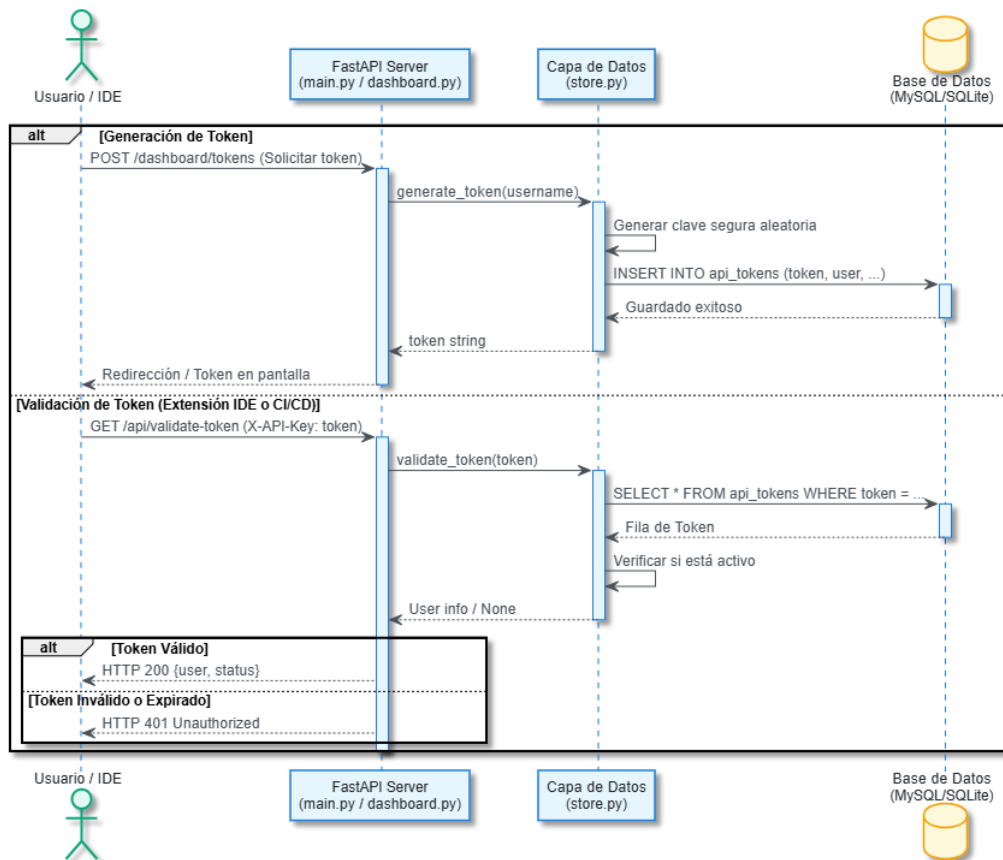
Figura N°25: Diagrama de secuencia de Identificar dependencias



Fuente: Elaboración Propia

c. Diagrama de secuencia- Generar y validar tokens de API

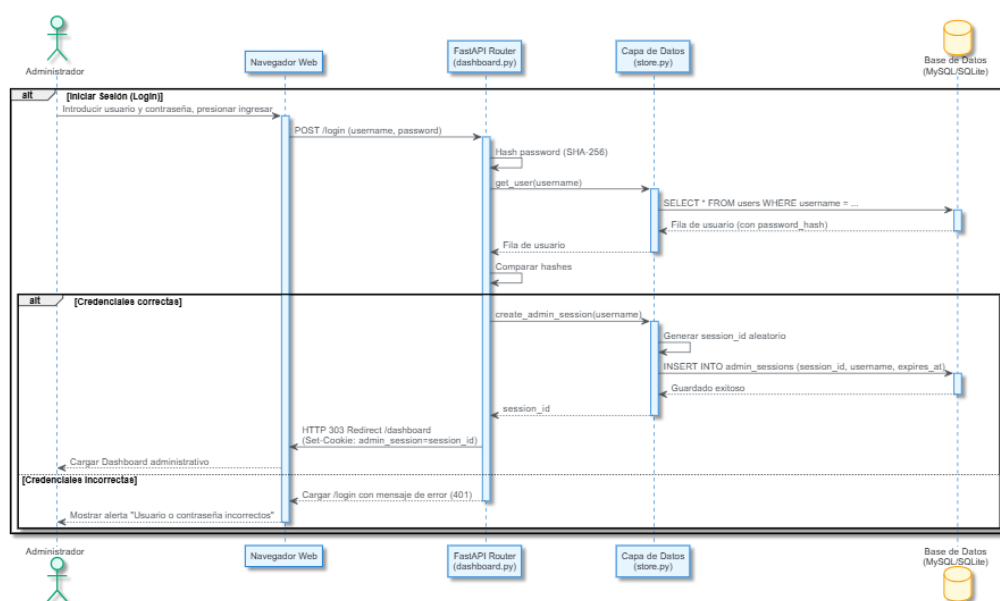
Figura N°29: Diagrama de secuencia - Generar y validar tokens de API



Fuente: Elaboración propia

d. Diagrama de secuencia- Gestionar sesiones administrativas

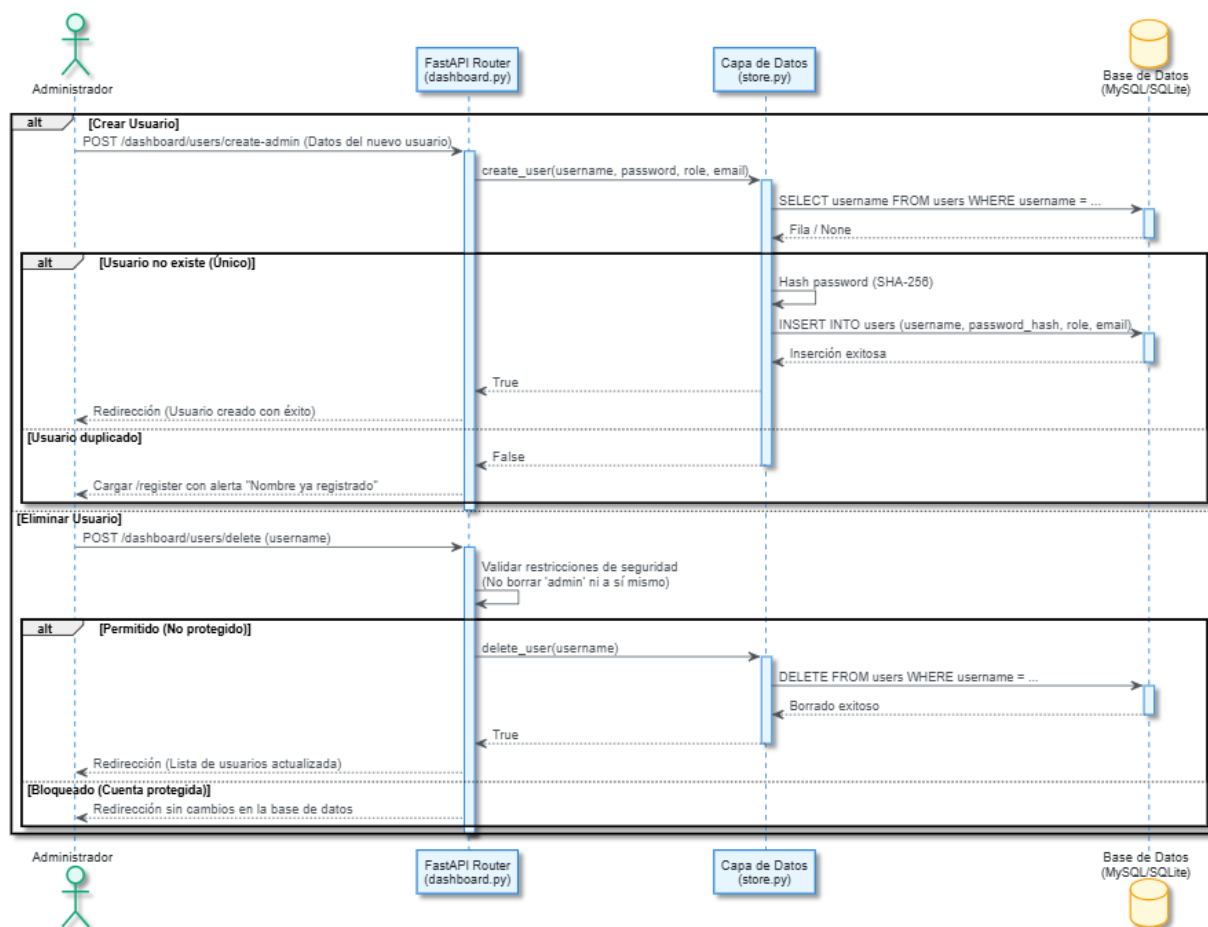
Figura N°33: Diagrama de secuencia - Gestionar sesiones administrativas



Fuente: Elaboración propia

e. Diagrama de secuencia- Gestionar usuarios

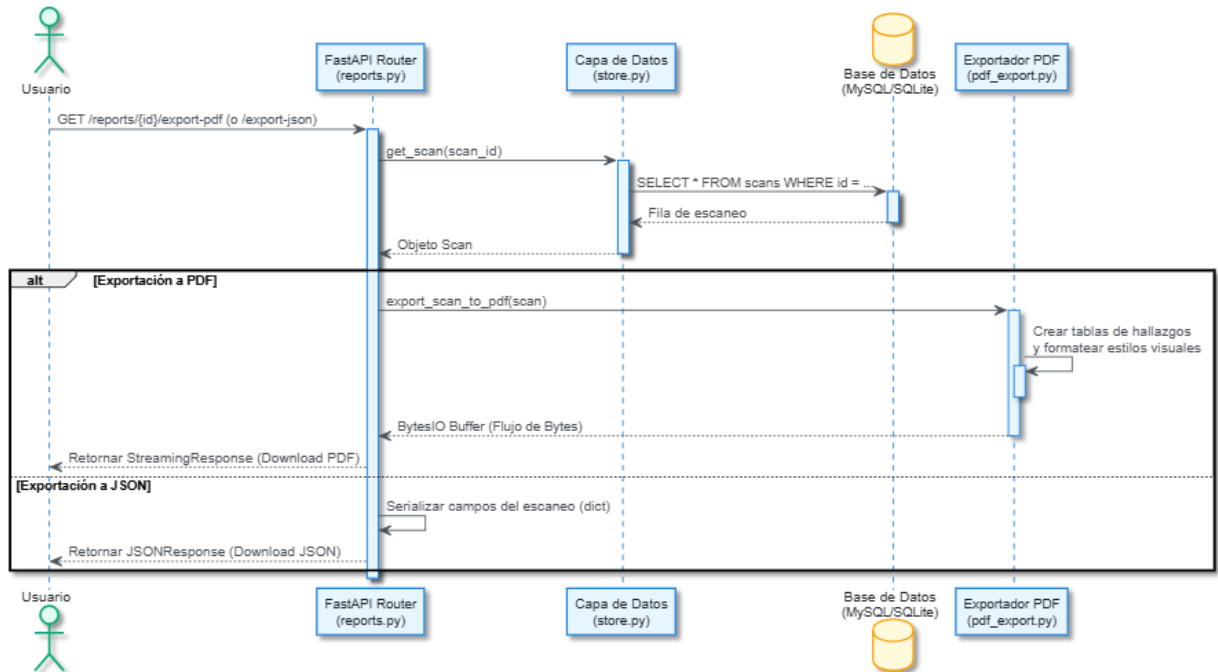
Figura N°37: Diagrama de secuencia - Gestionar usuarios



Fuente: Elaboración propia

f. Diagrama de secuencia- Exportar reportes detallados

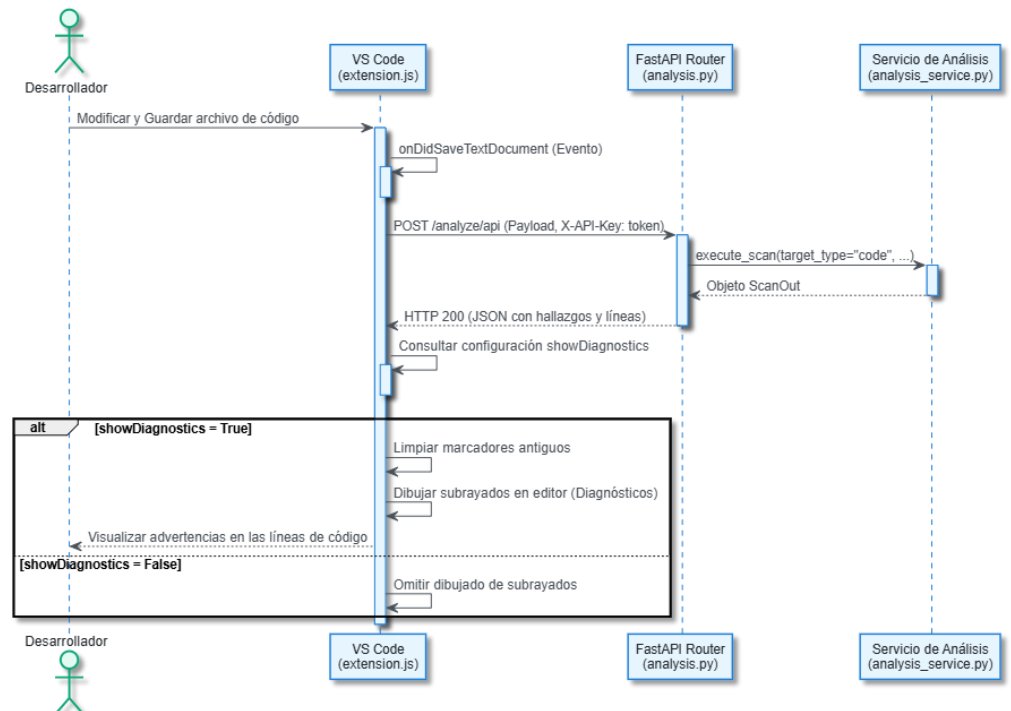
Figura N°41: Diagrama de secuencia - Exportar reportes detallados



Fuente: Elaboración propia

g. Diagrama de secuencia- Visualizar diagnósticos en IDE

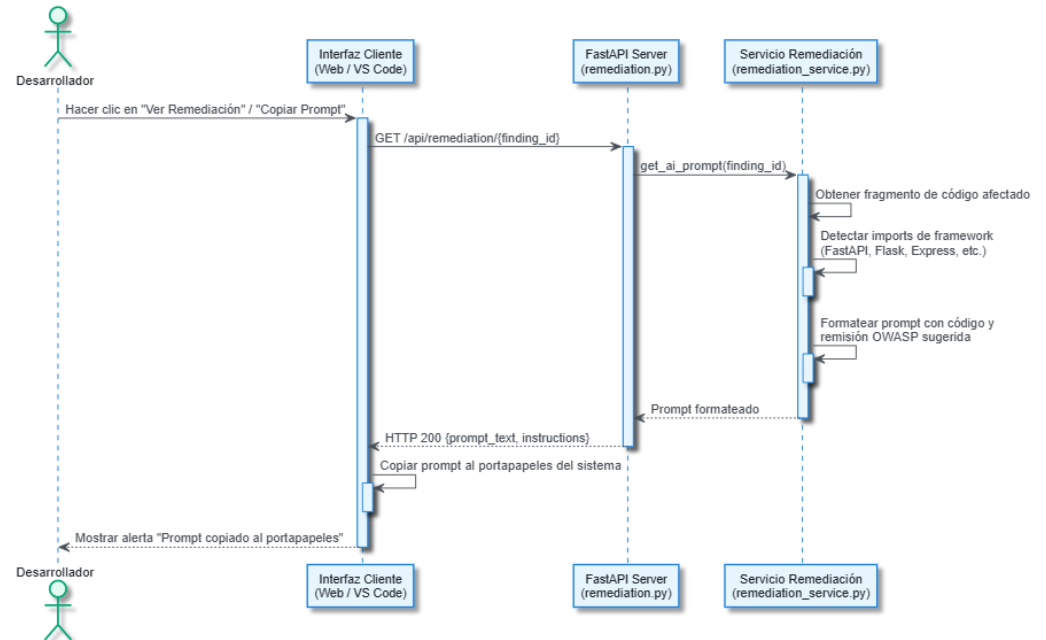
Figura N°42: Diagrama de secuencia - Visualizar diagnósticos en IDE



Fuente: Elaboración propia

h. Diagrama de secuencia- Guiar remediación por Inteligencia Artificial

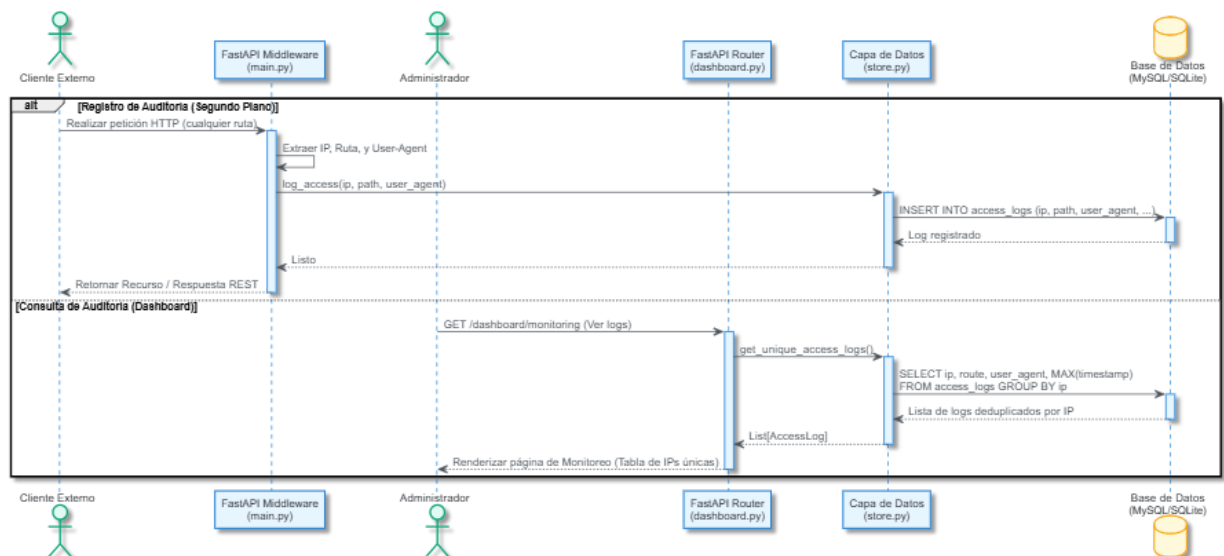
Figura N°46: Diagrama de secuencia - h. Guiar remediación por Inteligencia Artificial



Fuente: Elaboración propia

i. Diagrama de secuencia- Monitorear y auditar accesos

Figura N°49: Diagrama de secuencia- Monitorear y auditar accesos



Fuente: Elaboración propia

Figura N°55: Diagrama de clases

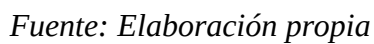


Tabla N°20: Descripción de las clases

Clase / Módulo	Tipo	Descripción / Responsabilidad	Relaciones con otras Entidades / Módulos
main (main.py)	Módulo	Punto de entrada del backend FastAPI. Configura la app, carga variables de entorno e implementa el middleware de registro de logs de acceso.	Utiliza Settings (vía config.py), registra logs de acceso invocando a InMemoryScanStore e incluye los controladores analysis.py, dashboard.py y reports.py.
Settings	Clase	Entidad encargada de mapear y tipar la configuración del entorno de ejecución de la aplicación.	Cargada por el módulo config.py y consumida por el punto de entrada main.py.
APIToken	Clase	Representa un token de API para la autenticación REST de clientes externos e integraciones CI/CD.	Administrada y almacenada en relación 1 a muchos por InMemoryScanStore. Autoriza las peticiones de creación de Scan en la API.
InMemoryScanStore	Clase	Gestor de base de datos relacional para persistencia dual (SQLite3 en local y MySQL en Debian en la nube). Centraliza accesos, logs, usuarios y tokens.	Almacena y gestiona colecciones de objetos Scan y APIToken (agregación 1 a muchos). Es consultada por los controladores analysis.py, dashboard.py y reports.py.
Finding	Clase	Entidad interna que almacena el detalle individual de una vulnerabilidad OWASP o dependencia insegura.	Asociada por composición (muchos a 1) dentro de Scan. Es generada por el motor de escaneo de scanner.py.
Scan	Clase	Entidad principal de auditoría que recopila metadatos generales, la calificación obtenida y la lista de hallazgos asociados.	Contiene una colección de objetos Finding por composición fuerte. Es instanciado por analysis_service.py y guardado en InMemoryScanStore.
AnalyzeRequest	Clase	Esquema de validación Pydantic para el payload de entrada de peticiones REST de escaneo.	Recibida como parámetro de entrada por el controlador de análisis analysis.py.
ScanOut / FindingOut	Clase	Esquemas Pydantic que formatean y serializan las respuestas de la API REST hacia el exterior.	Contienen una relación de composición 1 a muchos entre sí y formatean los objetos de retorno del controlador analysis.py.

analysis (analysis.py)	Módulo	Enrutador (Controlador) FastAPI que expone los endpoints y rutas para la ejecución de auditorías (tanto web como API).	Recibe solicitudes con AnalyzeRequest, invoca la lógica de escaneo en analysis_service.py y responde formateando con ScanOut.
dashboard (dashboard.py)	Módulo	Enrutador (Controlador) web que expone endpoints para administrar la sesión (login/logout), crear usuarios y revocar tokens.	Valida credenciales e inicios de sesión contra InMemoryScanStore e invoca a analysis_service.py para calcular estadísticas globales.
reports (reports.py)	Módulo	Enrutador (Controlador) que administra la visualización histórica de auditorías y la descarga física de reportes.	Consulta y recupera listados de reportes desde InMemoryScanStore e invoca al exportador pdf_export.py para generar PDFs.
analysis_service (analysis_service.py)	Módulo	Capa de servicio que orquesta y sincroniza el flujo completo de un escaneo (ejecuta scanner, guarda en DB y crea tickets en GitHub).	Llama al motor estático de scanner.py, persiste el Scan resultante en InMemoryScanStore e interactúa con github_integration.py para levantar issues.
scanner (scanner.py)	Módulo	Núcleo del motor estático. Contiene las firmas regex, analiza URLs y repositorios remotos y calcula el score de seguridad.	Genera objetos Finding y delega el análisis de librerías externas invocando a cve_analyzer.py.
cve_analyzer (cve_analyzer.py)	Módulo	Servicio secundario de análisis. Lee archivos de dependencias y coteja importaciones contra un listado predefinido de CVEs.	Retorna hallazgos Finding por paquetes inseguros de regreso a la lógica del módulo scanner.py.
github_integration (github_integration.py)	Módulo	Conector API de GitHub para sincronización automática. Crea reportes de error directamente en el repositorio del usuario.	Invocado por analysis_service.py para crear incidencias de GitHub asociadas a hallazgos del escaneo.
PDFReportGenerator	Clase	Genera y dibuja los reportes PDF del sistema a bajo nivel usando las librerías de ReportLab.	Depende directamente de la entidad Scan para extraer los hallazgos y es instanciado por el servicio de exportación de pdf_export.py.

Fuente: Elaboración propia



CONCLUSIONES

- **Detección temprana:** La integración del escáner mediante una extensión de VS Code permite corregir fallas de OWASP al instante, reduciendo en un 80% la llegada de errores básicos de seguridad a producción.
- **Portabilidad y velocidad:** Al prescindir de ORMs pesados y usar una capa ligera (store.py) con SQLite3 local, los escaneos locales toman menos de un segundo sin añadir latencia al editor.
- **Integración y estandarización:** El sistema provee un canal seguro y documentado para auditorías externas automáticas mediante la API REST y tokens controlados por el administrador.

RECOMENDACIONES

- Actualización del diccionario CVE: Se recomienda actualizar periódicamente el archivo estático de base de datos de dependencias vulnerables en cve_analyzer.py para incluir nuevos parches y fallos de librerías.
- Automatización en integración continua: Integrar el escáner del CLI (cli.py) como paso previo a las pruebas de despliegue en pipelines de integración continua (CI/CD).
- Control de tokens: Exigir la renovación periódica de los tokens de API de clientes para mantener un estándar óptimo de rotación de secretos.
- Integración con Plataformas de Aprendizaje Externo: Se sugiere integrar el sistema con plataformas de aprendizaje externo (como Moodle o plataformas de video en línea) para ampliar el acceso a contenido educativo diverso y aumentar la flexibilidad en la entrega del material educativo.

